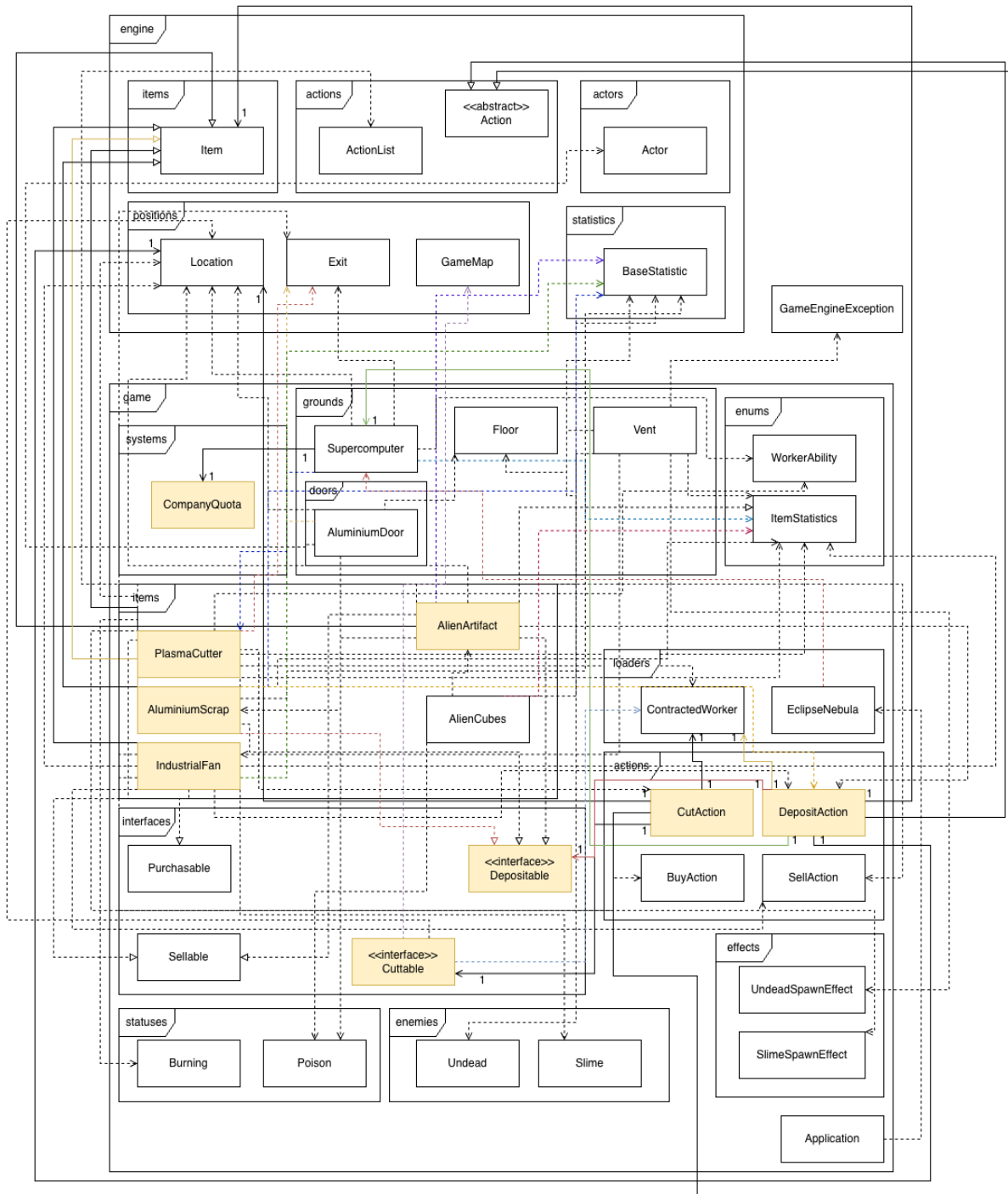


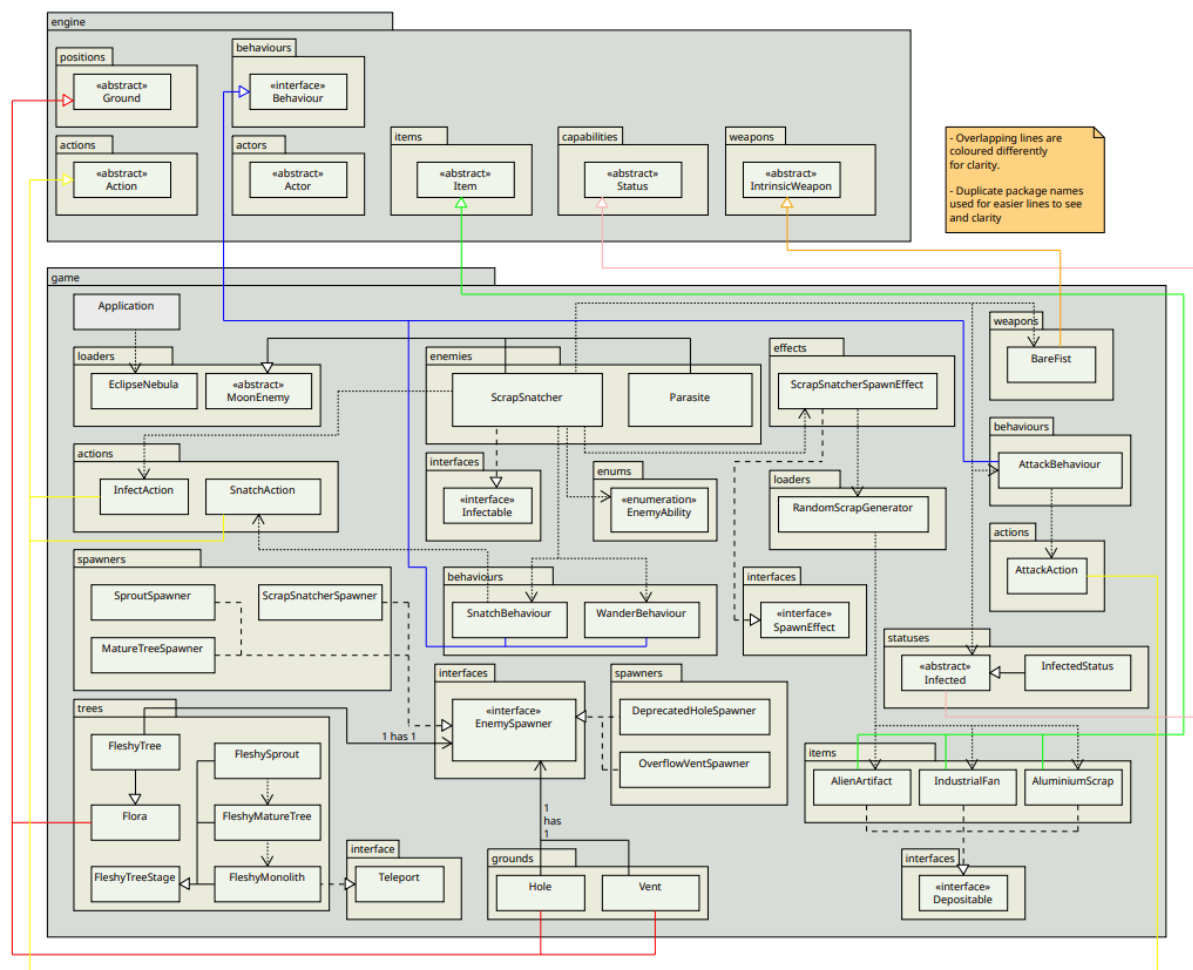
FIT2099 Assignment 3 Design Documentation

Design Diagrams

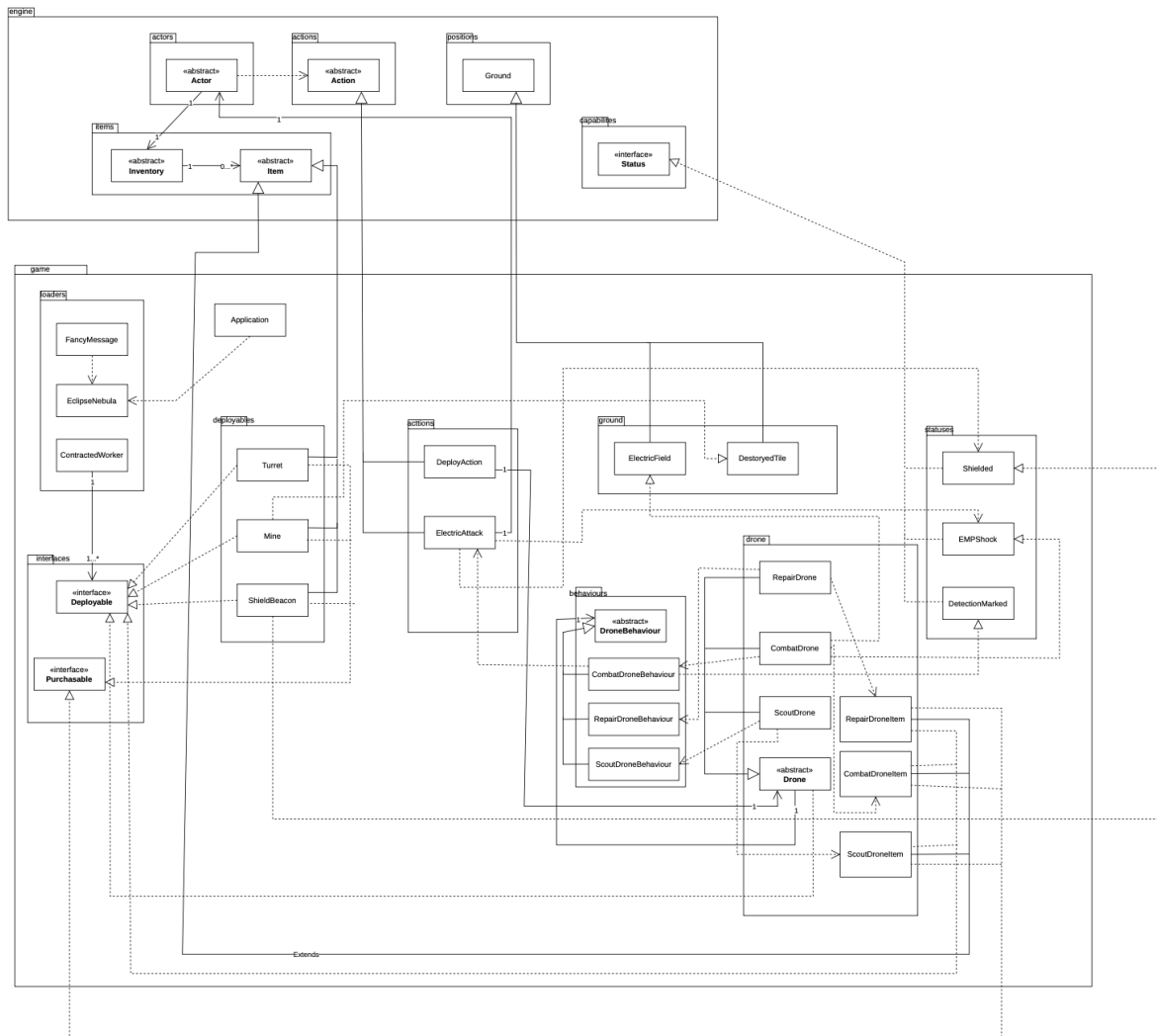
REQ1 UML Class Diagram:



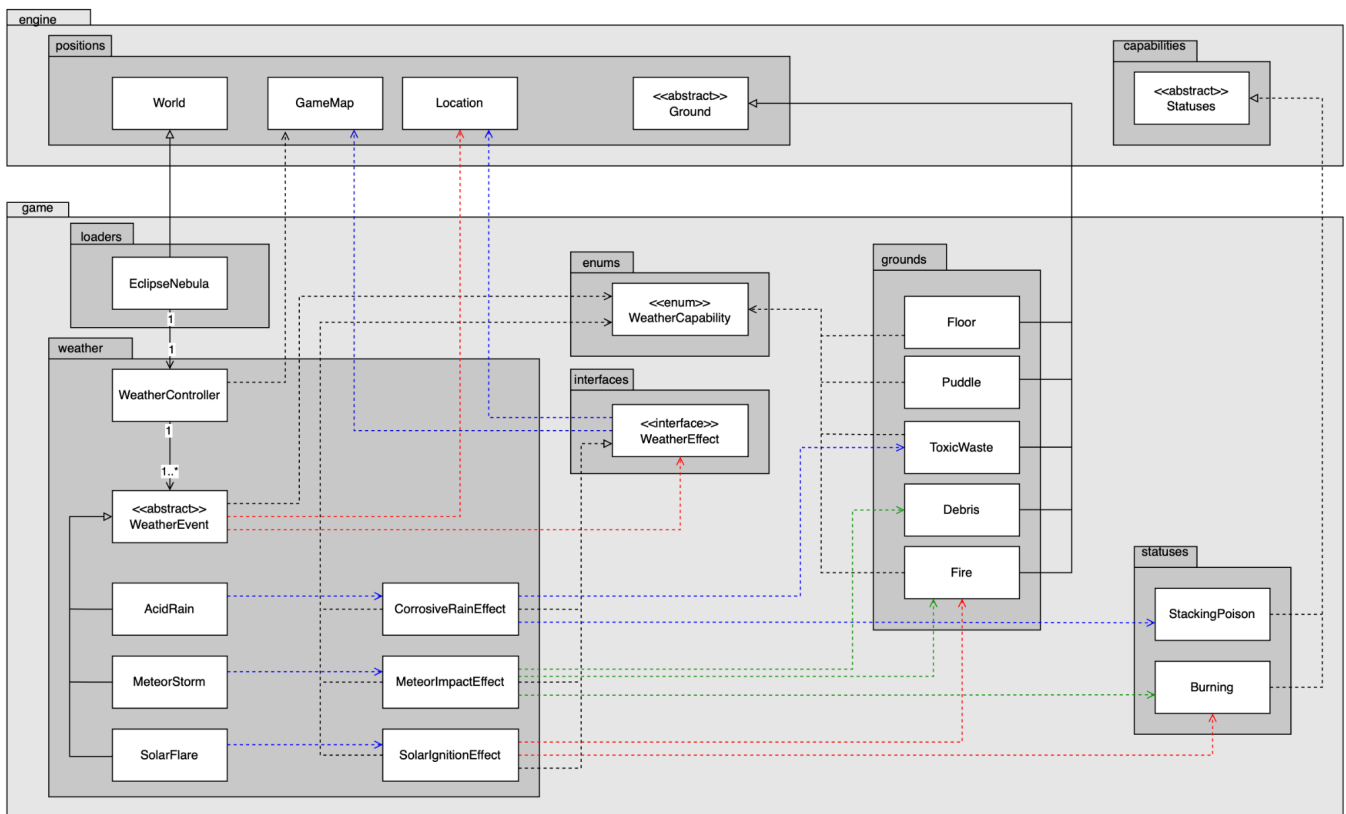
REQ2 UML Class Diagram:



REQ3 UML Class Diagram:



REQ4 UML Class Diagram:



REQ5 UML Class Diagram:

Design Rationales

REQ1: Quota

a. Implementation of the quota cycle

Design 1: Create concrete CompanyQuota class that doesn't inherit any of the abstract or interface class and implement its features inside the concrete class itself such as company rank, credit and a time limit for quota cycle

Pros	Cons
Simpler & flexible, able to only implement the functionality as all logics are in one place in the CompanyQuota concrete class.	Future extension of the similar concrete classes will no longer share methods that implement the same feature and have duplicated codes unless you create a new abstract or interface class
Single Responsibility Principle is kept here as CompanyQuota concrete class handles the quote cycle with rank, company credit and a time limit for each supercomputer	

b. Implementation of the plasma cutter

Design 1: Create concrete class PlasmaCutter that inherits Item abstract class from engine and Purchasable interface class

Pros	Cons
Inheriting from abstract Item class allows reusability of existing code and less repeated code and clearer implementation of PlasmaCutter.	Inheriting an Item abstract class introduces tight coupling between their child concrete classes and the Item abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.
Future extension of common methods only needs to be added to the abstract super class, child classes will inherit the methods with no need to modify child classes.	

Design 2: Add PURCHASABLE enum constant in ItemStatistics enum class to differentiate plasma cutter as purchasable and add the PURCHASABLE statistic to the plasma cutter and inherits Item abstract class from engine

Pros	Cons
Inheriting from abstract Item class allows reusability of existing code and less repeated code and clearer implementation of PlasmaCutter.	Inheriting an Item abstract class introduces tight coupling between their child concrete classes and the Item abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.
Future extension of common methods only needs to be added to the abstract super class, child classes will inherit the methods with no need to modify child classes.	There is no more Purchasable interface class as well as abstract class that will share between purchasable items so there will be inconsistency between purchasable items.

c. Implementation of the cutting action with the plasma cutter for aluminum door, vent and alien cubes

Design 1: Overrides allowableActions method to check the surroundings to find AluminumDoor and Vent as well as worker's inventory for AlienCubes in PlasmaCutter class. AluminiumDoor, Vent and AlienCubes will have CUTTABLE statistics and inherit the Cuttable interface class. CutAction concrete class inherits Action abstract class from engine

Pros	Cons
Utilizes the existing method hasStatistic from ground and item abstract class to check them in PlasmaCutter allowableActions to prompt menu options to the user whether they want to cut it or not.	Duplicated code for recognizing Cuttable ground or items as we add cuttable statistics as well as inheriting Cuttable interface class which violates Don't Repeat Yourself Principle.
Inheriting Action abstract class for CutAction concrete class makes consistent between all Actions in the system.	

Design 2: Overrides allowableActions method to check the surroundings to find AluminumDoor and Vent as well as worker's inventory for AlienCubes in PlasmaCutter class. AluminiumDoor, Vent and AlienCubes will inherit the Cuttable interface class. CutAction concrete class inherits Action abstract class from engine

Pros	Cons
Reduced duplicated code since we do not add CUTTABLE statistics to those three.	Does not utilize the existing feature to distinguish amongst the items and grounds with statistics anymore.
Inheriting Action abstract class for CutAction concrete class makes consistent between all Actions in the system.	

d. Implementation of the new resources such as aluminum scrap, industrial fan and alien artifact that can be deposited or sold to the supercomputer

Design 1: Create Depositable interface with its methods as well as DEPOSITABLE enum constant and add statistics to those new resources correspondingly. Those new resources will be new concrete classes that inherit the Item abstract class and Sellable or Depositable interface classes.

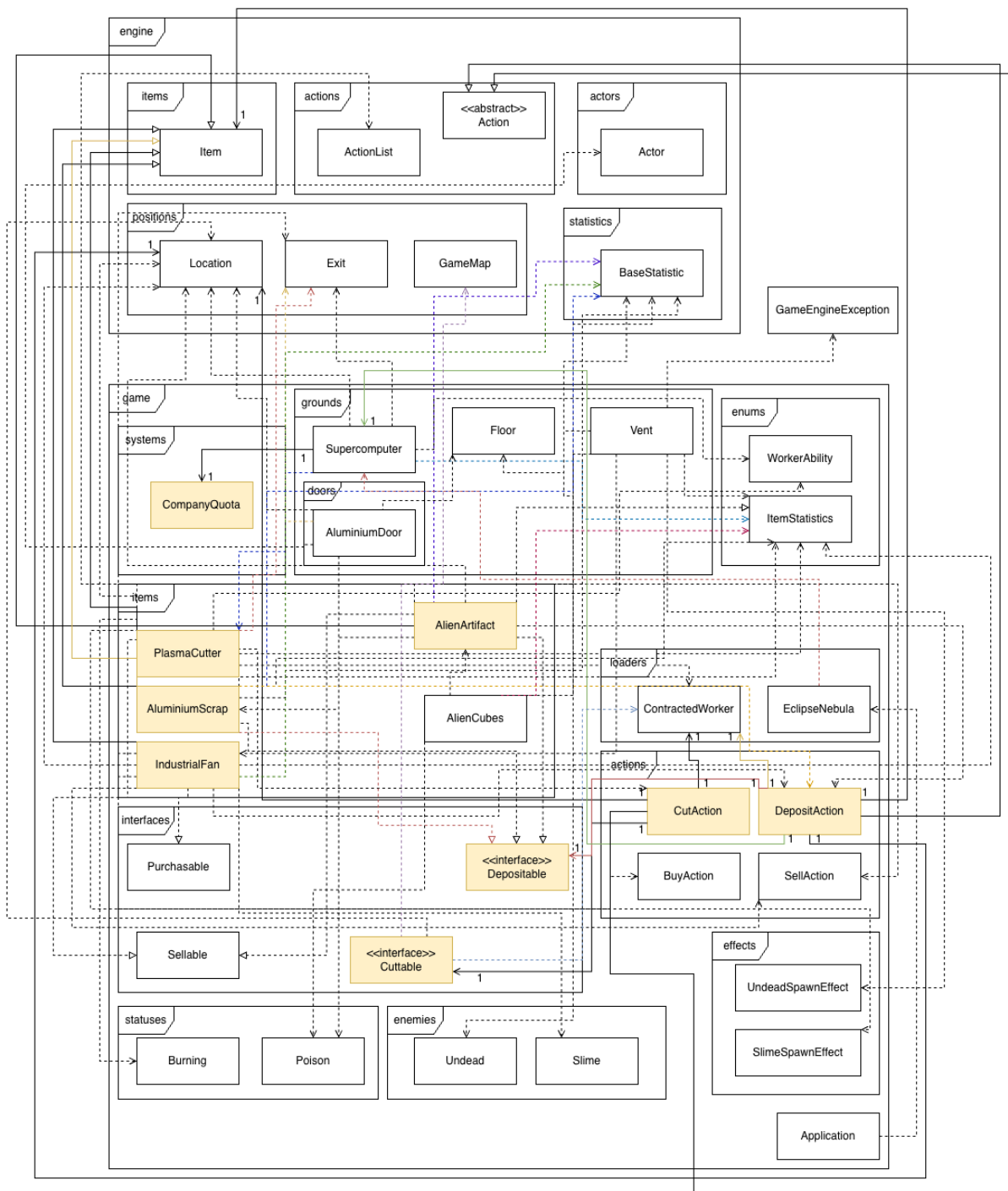
Pros	Cons
Inheriting the Depositable interface class for the new concrete classes makes it consistent between them in the system.	Duplicated code for recognizing Depositable items as we add DEPOSTIABLE statistics as well as inheriting Depositable interface class which violates Don't Repeat Yourself Principle.
Utilizes the existing method hasStatistic from an item abstract class to check whether they have the statistics we are looking for when we are dealing with the supercomputer so we can prompt the menu options to the user using it.	

Design 2: Create a Depositable interface with its methods. Those new resources will be new concrete classes that inherit the Item abstract class and Sellable or Depositable interface classes.

Pros	Cons
Inheriting the Depositable interface class for the new concrete classes makes it consistent between them in	Duplicated code for recognizing Depositable items as we add DEPOSTIABLE statistics as well as

the system.

inheriting Depositable interface class which violates Don't Repeat Yourself Principle.



The diagram above shows the final design of requirement 1.

CompanyQuota concrete class does not inherit any abstract or interface class since it has no common idea from the existing game engines or interfaces. It manages the quota cycle for company rank, credit and a time limit for each supercomputer as described in the requirement 1. PlasmaCutter concrete class inherits from Item abstract class from the game

engine as well as Purchasable interface class since we can buy it from the supercomputer and it's one of the items that cuts the AluminiumDoor, Vent and AlienCubes. The way it handles the cut action is to override its parent's item's method allowableActions in the PlasmaCutter concrete class that it checks the surroundings of the worker as well as the inventory of the worker and add the menu options to prompt using its existing methods that handles it. AluminiumDoor, Vent and AlienCubes will now have cutBy methods since they will be inheriting Cuttable interface that requires cutBy method and it will perform the effect that happens after a worker cuts them by PlasmaCutter. Three new resources such as AlienArtifact, AluminumScrap and IndustrialFan will be sold or deposited to the supercomputer so they inherit Sellable or Depositatable interface class based on the requirement 1 description as well as SELLABLE and DEPOSITABLE statistics to make use of the 'hasStatistic' from existing methods in the parents class and they will implement the methods that their interface class requires to such as onSell, onDeposit etc to implement the effect after selling or depositing to the supercomputer.

REQ2: The Scrap Snatcher

A. Choosing Scrap Snatcher structure

Design 1: Create ScrapSnatcher as a concrete class that directly extends to the engine's base code Actor class, without any abstract enemy class.

Pros	Cons
The class contains what it needs for this specific enemy, preventing it from inheriting unwanted behaviours from a superclass (KISS)	If we use this for more multiple enemies in the future, this could violate DRY, and will become less extensible.
	It will also have poor maintainability as if new methods are needed, we would have to manually code all the standalone enemy classes.

Design 2: Extend to pre-existing MoonEnemy abstract class for the blueprint for ScrapSnatcher.

Pros	Cons
There is a strong code reuse when having an abstract class as the base. So ScrapSnatcher will inherit the common methods similar to other enemies.	Extending an abstract class introduces tight coupling between the concrete class and the abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.
The engine, spawners, and other actors can treat the ScrapSnatcher generically as a MoonEnemy (LSP).	
This helps with future extensibility, for example if all enemies will now drop gold upon death we can simply add the logic to MoonEnemy instead of doing one by one in all enemy classes.	

B. ScrapSnatcher Infection state and its behaviours

Design 1: InfectAction directly access the ScrapSnatcher object, calls a getter for its behaviours and manually removes SnatchBehaviour to insert AttackBehaviour

Pros	Cons
------	------

The code will be direct and the execution for behaviour state changes will be immediate.	This might have feature envy as InfectAction wants to look deeper for the internal structure for ScrapSnatcher.
	Highly coupled.
	There may be a high connascence of execution as the InfectAction must know exactly how ScrapSnatcher prioritises its behaviours. If ScrapSnatcher behaviour structure changes, it might break the action code execution.

Design 2: Uses the engine's capability system. InfectAction and SnatchBehaviour applies the EnemyAbility.IS_RABID to the target. The internal systems will check the hasAbility() before executing.

Pros	Cons
This promotes high cohesion and lower coupling as InfectAction and SnatchBehaviour is still responsible for applying its own logic but also make it unaware of how it affects its target	Must evaluate conditions every turn which adds more processing to the code and playTurn loop.
If a new enemy comes up that uses this classes, we simply add it and this improves future extensibility.	
It also eliminates hard type checking and lowers coupling to connascence of name where it shares the enumeration.	

C. Loot explosion effect for ScrapSnatcher spawning (deposable items: AluminiumScrap, IndustrialFan, AlienArtifact. etc)

Design 1: Hardcode the instantiation of items like new AluminiumScrap(), new Industrialfan(), etc. into Hole and Vent classes after ScrapSnatcher is spawned.

Pros	Cons
This is a quick and easy code solution to cover the loot explosion logic.	Ground tiles like Hole and Vent should not depend on item classes (DIP).

	Reduces code extensibility and maintainability as every time a new item that is depositable is created. We have to add more of it into Hole and Vent.
--	---

Design 2: Uses a new method RandomScrapGenerator to generate random depositable items, and inject the logic into ScrapSnatcherSpawnEffect that implements a SpawnEffect interface. This SpawnEffect provides a method call applySpawnEffect(spawn, location).

Pros	Cons
Spawners interact with the SpawnEffect instead of concrete item logic (DIP). We also do not repeat the code multiple times.	Introduces more classes to the system.
Improves code maintainability.	

D. Growth logic in flora stages (Sprout, Mature, Monolith) and its ecosystem

Design 1: Create new classes for Sprout and Mature for deprecated map specifically. Hardcode their turn timers and math values directly into their tryGrow() method.

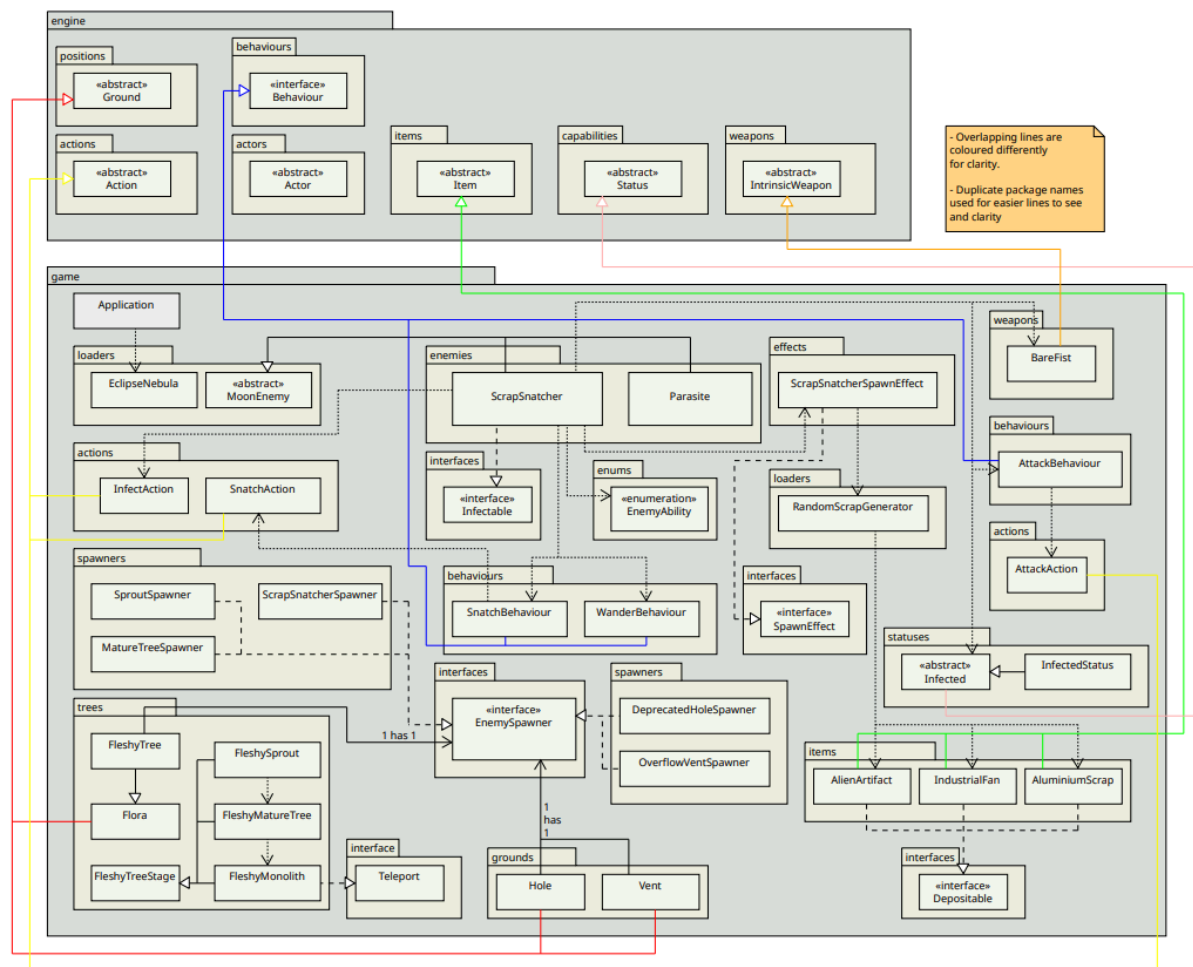
Pros	Cons
This is a quick and easy code solution to cover the growth logic.	Less code maintainability.
	More grounds will be added as we already have a Sprout, Sapling and Mature in overflow map. Adding 3 more extra classes of the same code is not helping with code extensibility.

Design 2: Inject a GrowingBehaviour<T> into the tree stages. The classes pass their turns and probabilities of the chances it grows into the constructor.

Pros	Cons
Eliminates magic numbers such as chances of growing and the turn it takes defined as parameters.	Requires dependency injection which makes the code more complex.

It also allows for just a FleshyTree, FleshyMatureTree and FleshyMonolith that can be used for both maps, no need to add extra classes.

Improves the code extensibility and future developments



The diagram shows the final design of requirement 2, which utilizes designs of part A, B, C, D with Design 2 stated above to implement a ScrapSnatcher enemy with complex infection state behaviours, a dynamic loot drop system when ScrapSnatcher spawns, and lastly a progressive multiple stage flora system.

The new ScrapSnatcher class will extend the abstract MoonEnemy class. We abstract these classes to avoid repetitions in the code (DRY), as the ScrapSnatcher shares similar common attributes and methods with other enemies. Extending these classes allows subclasses to inherit these functionalities without changing behaviours or abilities expected by the base class smoothly. This ensures it integrates well with the engine's actor system (LSP).

Utilising the engine's capability system which has status for the infection state allows InfectAction to simply add EnemyAbility.IS_RABID to the target. This gives InfectAction a

strict responsibility of inflicting the infected status rather than changing its target's internal system. This promotes segregation of responsibilities between classes and methods (SRP). SnatchBehaviour also evaluates the hasAbility() condition before executing, this avoids type-checking and allows future code usage to utilise the infection logic smoothly without needing to modify the core action of it (OCP).

The ScrapSnatcherSpawnEffect implements the SpawnEffect interface which provides a clear contract for spawn mechanics and ScrapSnatcherSpawnEffect uses the method applySpawnEffect(spawn, location) method. Using an interface ensures that the spawning system relies on abstraction rather than being concrete effect classes to trigger spawning changes (DIP). Moreover, introducing RandomScrapGenerator centralises the logic for generating the depositable items (AluminiumScrap, IndustrialFan, AlienArtifact, etc). Since i transfer the generation depositable item logic out of the effect class and into a generator class, this isolates and segregates the game's responsibilities (SRP).

Lastly, for the flora ecosystem (Sprout, Mature, Monolith), we inject a GrowingBehaviour<T> into the tree stages for the growth logic. By passing the specific turn counts and probabilities of the math logic directly into the constructor, we can improve code reusability by preventing the same turn count and math values from being duplicated code across every concrete tree stages (DRY). Since we relocated the calculation for growing into GrowingBehaviour, this prevents the tree classes from managing complex math calculations and ensuring they focus on representing the specific tree stages in the game map (SRP).

REQ3: AUTONOMOUS DEPLOYMENT & TACTICAL SUPPORT SYSTEM

This feature introduces an Autonomous Deployment & Tactical Support System that allows contracted workers to deploy drones, traps and support structures that continue operating independently after placement. Instead of relying on direct player actions each turn, deployed entities execute autonomous behaviours or environmental logic that influence combat, survivability over time. The design goal was to separate deployment, autonomous decision-making and environmental interaction into distinct abstractions to support scalability and maintainability.

1. Deployable as Interface vs Abstract Class

The Deployable abstraction defines a shared contract for all deployable items such as drones, turrets, mines and support structures. It ensures that all deployable entities can be handled uniformly through DeployAction.

Design 1: Deployable as an abstract class

Pros	Cons
Could provide shared deployment state or helper methods (e.g. default validation logic).	Forces all deployables into a single inheritance hierarchy, preventing them from extending other base classes (limiting flexibility).
Common deployment logic (e.g. inventory removal checks) could be centralised.	Many deployables have completely different activation rules (immediate spawn vs delayed activation), making a shared base class unnecessarily restrictive.
	Encourages inheritance coupling even when behaviours are unrelated.

Design 2: Deployable as an interface

Pros	Cons
No inheritance restriction. Items already extend Item and can still implement Deployable.	No shared helper methods, requiring some repeated logic across deployables.
Defines a clean contract (deploy + getDeployAction) without enforcing structure.	Each class must implement its own deployment logic explicitly.

Supports DIP: DeployAction depends only on Deployable, not concrete items.	Each class must implement its own deployment logic explicitly.
Supports DIP: DeployAction depends only on Deployable, not concrete items.	

Decision

Design 2 was chosen because deployables vary significantly in activation behaviour (instant placement for Turret and Shield Beacon vs delayed triggering for Mine and drone spawning items). Forcing a shared abstract class would introduce unnecessary constraints and reduce flexibility.

The interface approach ensures that DeployAction operates purely on an abstraction, allowing new deployable types to be added without modifying existing systems (OCP).

2. DroneBehaviour as Abstract Class

DroneBehaviour defines the autonomous decision-making lifecycle for all drone entities (Combat, Scout, Repair).

Design 1: DroneBehaviour as interface

Pros	Cons
Maximum flexibility for different behaviour implementations.	Requires duplication of shared logic if future behaviours need common utilities.
Fully decouples behaviour implementations.	No enforced structure for drone decision flow.
	Each behaviour would independently define execution rules, increasing inconsistency risk.

Design 2: DroneBehaviour as abstract class

Pros	Cons
Enforces a consistent decision contract via <code>getAction()</code> .	Introduces inheritance coupling.

Allows shared helper logic (e.g. scanning utilities or targeting patterns in future extensions).	Cannot extend another class.
Ensures all drones follow the same decision-making structure (Template Method readiness).	

Decision

Design 2 was chosen because all drones follow a consistent execution cycle: scan then select target then return action. Even though current implementations are simple, the abstraction prepares the system for future shared targeting utilities without duplicating logic across behaviours.

3. Separation of Drone Entity and Drone Item

Design 1: Single class combining item + drone entity

Pros	Cons
Fewer classes and simpler structure initially.	Violates Single Responsibility Principle (SRP).
Immediate spawning and behaviour logic in one place.	Deployment logic and autonomous behaviour become tightly coupled.
	Harder to manage lifecycle transitions (item then active drone).

Design 2: Separate Item (Deployable) and Drone (Actor)

Pros	Cons
Clear separation between deployment phase and autonomous behaviour phase.	More classes required per drone type.
Item handles placement/activation logic only. Drone handles AI behaviour.	Requires coordination between item and spawned actor.
Supports extensibility (new drone behaviours without changing item logic).	Slightly more complex lifecycle flow.

Enables reuse of DroneBehaviour across multiple drone types.	
--	--

Decision

Design 2 was chosen because deployment and autonomous behaviour are fundamentally different concerns. Items represent pre-deployment inventory objects, while drones represent active autonomous actors. Separating them ensures SRP is maintained and allows behaviours to evolve independently of deployment logic.

4. ElectricField + EMPShock Interaction Model

Design 1: Direct hardcoded effects inside each system

Pros	Cons
Simple to implement quickly.	Strong coupling between terrain, status, and actors.
No abstraction overhead.	Effects cannot be reused or extended cleanly.
Supports extensibility (new drone behaviours without changing item logic).	Violates OCP due to repeated conditional logic across systems.

Design 2: Capability-based status + terrain system

Pros	Cons
EMPShock and ElectricField operate via shared status/ability system.	Requires consistent status checking across multiple systems.
ElectricField applies effects independently via tick().	Slight duplication of status checks in different environments.
EMPShock integrates with DroneBehaviour lifecycle (disables actions cleanly).	

Supports emergent interactions (ElectricField then EMPShock then drone shutdown).	
--	--

Decision

Design 2 was chosen because environmental hazards and status effects interact across multiple systems (terrain, actors, drones). A decoupled status-based system allows these interactions to scale without requiring direct dependencies between classes.

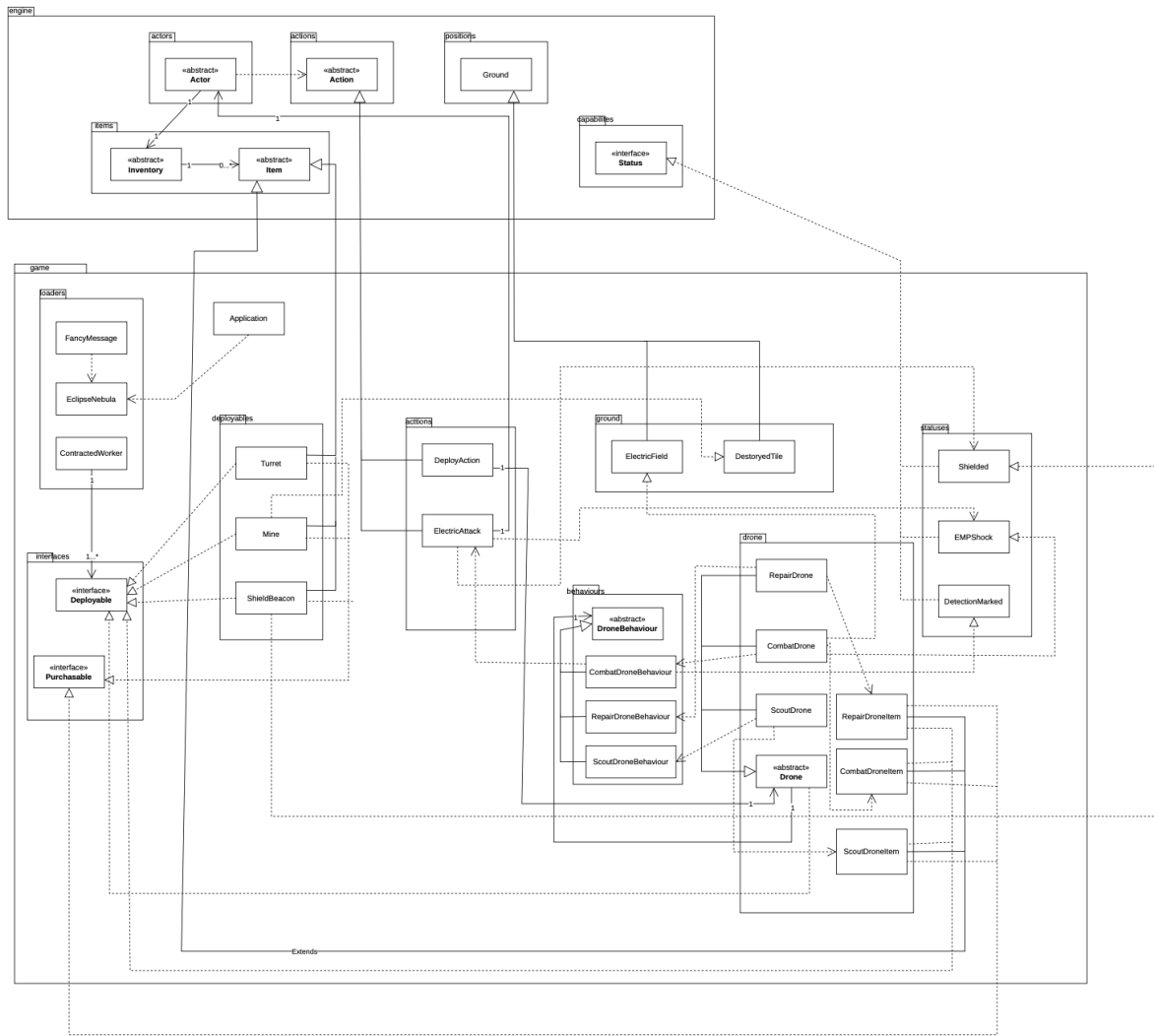
Overall Architecture Summary

The system is structured around two core abstractions:

- **Deployable (Interface)**
Used by items that can be deployed into the world (drones, turrets, mines, beacons). Enables uniform handling via DeployAction.
- **DroneBehaviour (Abstract Class)**
Defines autonomous decision-making logic for deployed drone actors. Each drone type specialises behaviour while maintaining a shared execution contract.

Limitation

A minor design limitation is that item classes directly instantiate their corresponding drone actors (e.g. CombatDroneltem to new CombatDrone()). This introduces a mild dependency between item and concrete drone class. This was retained for simplicity, but could be improved using a factory pattern to further decouple deployment logic.



The UML diagram shows two new abstractions: Deployable (interface) and DroneBehaviour (abstract class). The Deployable interface provides a common deployment contract that is implemented by six concrete deployable entities: CombatDroneItem, ScoutDroneItem, RepairDroneItem, Mine, Turret, and ShieldBeacon. This allows all deployable units and structures to be managed uniformly through DeployAction while allowing each class to define its own deployment behaviour.

The DroneBehaviour abstract class encapsulates autonomous decision-making for drones and has three concrete implementations: CombatDroneBehaviour, ScoutDroneBehaviour, and RepairDroneBehaviour. The abstract Drone class maintains a composition relationship with DroneBehaviour, delegating its turn logic through the behaviour object. This allows new drone behaviours to be added without modifying the Drone class itself.

Three concrete drone entities (CombatDrone, ScoutDrone, and RepairDrone) inherit from Drone and are instantiated by their corresponding deployable item classes after deployment. The item classes act as deployment proxies, delaying drone creation until deployment conditions are satisfied. This separates inventory management from autonomous actor behaviour.

Supporting classes EMPShock, DetectionMarked, and Shielded implement the retrofitted Status interface and provide the status-effect subsystem. These statuses are applied by drones, deployables and environmental hazards to influence combat and autonomous behaviour. ElectricAttack serves as a specialised action used by Combat Drones to inflict damage and apply EMPShock, while DeployAction provides the common execution mechanism for all deployable entities.

The environment subsystem consists of ElectricField and DestroyedTile, both extending the engine Ground abstraction. ElectricField applies EMPShock and environmental damage to actors occupying affected tiles, while DestroyedTile temporarily modifies map traversal following Mine detonation. These ground classes allow deployables to influence the battlefield beyond direct actor interactions.

ContractedWorker is retrofitted to interact with the deployment system through Deployable and Purchasable abstractions, while EclipseNebula depends only on the high-level deployable classes required for world initialisation. This keeps game setup largely isolated from the implementation details of drone behaviours, status effects and environmental hazards.

REQ4: CREATIVE MODE - Weather System

Overview

This feature introduces a Dynamic Weather System that actively modifies the game map each turn. A WeatherController cycles through three weather events (AcidRain, MeteorStorm, and SolarFlare). Each of which targets specific ground tiles via a capability-based query system and delegates all world mutation to a paired WeatherEffect. The core design goal was to keep weather events decoupled from concrete ground types, and to separate the tile targeting from the mutation effects.

A. WeatherEvent as Abstract Class vs Interface

WeatherEvent needed to own the tick lifecycle which includes finding targets, applying the effect, and decrementing the duration, while only letting subclasses vary the targeting logic via findTargets().

Design 1: WeatherEvent as an interface

Pros	Cons
No inheritance coupling as any class could implement it freely.	Interface attributes in Java are public static final. Shared state like turnsRemaining and effect cannot be stored in an interface without breaking encapsulation.
More flexible for future classes that cannot extend a base class.	The tick() lifecycle would be duplicated across all three event classes (AcidRain, MeteorStorm, SolarFlare), violating DRY.
	No way to enforce a fixed tick() flow without relying on each subclass to call steps in the correct order.

Design 2: WeatherEvent as an abstract class

Pros	Cons
tick() is declared final, locking the lifecycle order and preventing subclasses from breaking it. This directly enforces the Template Method pattern.	Introduces inheritance coupling; changes to WeatherEvent propagate to all subclasses.
turnsRemaining, effect, scanMap(), and selectRandomSubset() are shared once in the base class, removing duplication across all three weather events (DRY).	Subclasses cannot extend another class in Java, limiting flexibility if a future event needed to extend a different base type.
findTargets() is abstract, forcing subclasses to define only which capability to scan for while the common lifecycle is stays the same.	

Design 2 was chosen because the tick() lifecycle is the same across all weather events. Making it final on an abstract class prevents subclasses from accidentally reordering steps such as decrementing turns before applying the effect. Design 1 would require each subclass to re-implement the same lifecycle, creating three identical tick() bodies, violating DRY and reduces maintainability. The tight coupling introduced by inheritance is acceptable here because all subclasses are specialisations of the same concept.

B. WeatherEffect as Interface vs Abstract Class

WeatherEffect needed to define the contract for world mutation.

Design 1: WeatherEffect as an abstract class

Pros	Cons
Could provide shared helpers (e.g. a report() method for console output) that all effects inherit.	Effects have no shared state or same lifecycle since there is nothing to enforce in a base class beyond the apply() method.
	Inheritance coupling would be introduced. Each effect mutates the world in a completely different way, so a shared base class adds inflexibility without reducing duplication.
	Prevent effect classes from extending another class if needed in future.

Design 2: WeatherEffect as an interface

Pros	Cons
Defines only the apply() method. Each effect is allowed to implement mutation however it needs to (ISP).	Shared helpers like console reporting cannot live in the interface, requiring either duplication or a separate utility.
WeatherEvent depends on the WeatherEffect interface rather than any concrete effect. This removes direct dependencies between event and effect classes (DIP).	
New effects can be added without modifying WeatherEvent or WeatherController (OCP).	

Design 2 was chosen because the three effects share no common behaviour; CorrosiveRainEffect spreads terrain, MeteorImpactEffect applies AoE and displaces actors, and SolarIgnitionEffect compounds fire spread. Forcing them to extend a common abstract class would introduce coupling where none is needed. An interface keeps the abstraction

minimal and focused (ISP), and ensures WeatherEvent depends on an abstraction rather than any concrete implementation (DIP).

C. WeatherCapability Enum for Ground Targeting

Design 1: Use instanceof to identify target ground types

Pros	Cons
No additional abstraction needed and is straightforward to implement.	Directly couples weather event classes to concrete ground classes (e.g. AcidRain importing Floor). This violates DIP.
	Adding a new ground type that should participate in weather requires modifying the event's findTargets() method, violating OCP.
	instanceof checks are a recognised code smell when used for type-based dispatch, as they signal that polymorphism should be used instead.

Design 2: WeatherCapability enum with hasAbility() queries

Pros	Cons
Weather events query only capabilities. AcidRain, MeteorStorm, and SolarFlare have no imports of ground types (DIP).	Ground classes must be modified to declare their capabilities, adding a small overhead when introducing new ground types.
A new ground type can participate in weather effects by declaring the relevant capability in its constructor, without changes to any weather class (OCP).	The capability enum must be kept consistent with what grounds declare; a mismatch (e.g. forgetting to add ACID_CORRODIBLE to a new floor variant) would exclude it.
Floor is retrofitted with ACID_CORRODIBLE, METEOR_TARGET, and FLAMMABLE, integrating existing ground into the weather system without modifying any event or effect class.	

Design 2 was chosen to eliminate all direct dependencies between weather events and concrete ground types. The capability enum acts as the abstraction layer where grounds declare what they are and weather events query for that capability without caring about the class behind it. This made it straightforward to retrofit Floor into the system and keeps all three event classes free of ground imports entirely.

D. Separation of Targeting and Mutation

Design 1: Single class per weather type (finds targets and applies effects)

Pros	Cons
Fewer classes to manage so initial structure is simpler.	Each class would have more than one responsibility: tile-targeting logic and mutation logic (effects), violating SRP
	Changing the spread logic of CorrosiveRainEffect would require modifying AcidRain, even though the targeting logic is unchanged.
	Testing targeting and mutation independently would require testing through the same class, making unit tests more difficult.

Design 2: Separate WeatherEvent (targeting) and WeatherEffect (mutation)

Pros	Cons
Each class has exactly one responsibility. WeatherEvent subclasses changes only if targeting logic changes; WeatherEffect implementations changes only if mutation logic changes (SRP).	More classes to maintain - six concrete classes instead of three.
Effects can be swapped or tested independently. For example, a different spread logic for acid rain requires only a new WeatherEffect, not a new WeatherEvent.	The relationship between a paired event and effect is implicit (set in the event constructor) rather than explicitly visible from a single class.
Targeting and mutation can be unit tested in isolation, improving test quality and coverage.	

Design 2 was chosen because the two concerns (which tiles to target and world mutation) change for different reasons. During development, the mutation logic for MeteorImpactEffect and SolarIgnitionEffect changed several times (adding knockback, adjusting spread logic) without any changes to MeteorStorm or SolarFlare. Keeping them separate meant those changes were contained to the effect class only, confirming the SRP benefit in practice.

Collapsing targeting and mutation into one class would introduce a connascence of algorithm where the tile-scanning logic and the mutation logic would need to change together within the same class. The separation reduces this to connascence of name at the

REQ5: A.P.I